Safety-based Termination Analysis With Transition Invariant Generation

Konstantin Britikov Martin Blicha Grigory Fedyukovich Natasha Sharygina

***Abstract*—Termination analysis of infinite-state systems is a fundamental problem in formal methods. Existing techniques are predominantly based on ranking function synthesis, which can be insufficient when the transition relation is too complex to admit a ranking function directly. We present a new approach to termination analysis based on the interpolation-based construction of well-founded transition invariants. By leveraging Craig interpolation, our approach guides transition invariant generation with respect to the conditions relevant to termination, enabling property-directed synthesis. Our technique extends the safety-based nontermination analysis, allowing it to prove both termination and nontermination within a unified framework. We implemented our approach and evaluated it on benchmarks from the Termination Competition (TermComp). Our tool demonstrates competitive performance against state-of-the-art termination provers, solving a significant number of instances not solved by any existing tool.**

I. Preliminaries

This section introduces the definitions and formalisms used in the paper. Logical formulas in our approach are defined over the theory of Linear Integer Arithmetic (LIA). We use x, x' to denote set of variables (unprimed and primed), where $x' = \{v' | v \in x\}$, x is a set of state variables, and x' is a set of next state variables. If there are variables of many different states in the formula we use superscript notation, e.g., $x^{(0)}, \ldots, x^{(n)}$. s, s' are used to denote the specific states in the system.

The formula defined over x is a state formula, and a formula over $x \cup x'$ is a transition formula. State formulas encode a set of states, and transition formulas define binary relations over the state. Using $Id(x, x')$ we denote a transition formula $x = x'$

a) Transition systems and Safety Problem: A *transition system* (TS) is defined as $T = \langle Init, Tr, x \rangle$, where $Init$ is a formula encoding the initial states of the system, Tr is a transition formula, which encodes the transition relation, and x is a set of system's variables. With respect to transition system, a *transition trace* is a formula encoding multiple consecutive transitions: $Tr^n(x^{(0)}, \ldots, x^{(n)}) = Tr(x^{(0)}, x^{(1)}) \wedge \cdots \wedge Tr(x^{(n-1)}, x^{(n)})$. We write $Tr^n(x^{(0)}, \ldots, x^{(n)})$ for this conjunction throughout the paper. A *safety problem* is defined as $P = \langle Init, Tr, Bad, x \rangle$ where Bad is a formula that represents states that violate the safety property.

Safety verification amounts to determining whether there exists a transition trace that reaches a Bad state from the initial states. If TS is safe, then there exists an inductive invariant $Inv(x)$ s.t. (1) $\forall x. Init(x) \rightarrow Inv(x)$, (2) $\forall x. Inv(x) \wedge Tr(x, x') \rightarrow Inv(x')$, and (3) $\forall x. Inv(x) \rightarrow \neg Bad(x)$. We use $safetyCheck(P)$ to denote a safety verification procedure that returns a tuple $\langle SAFE, Inv(x) \rangle$ if TS is safe ($Inv(x)$ is inductive invariant). Otherwise, if TS is unsafe it returns $\langle UNSAFE, Tr^n(x^{(0)}, \ldots, x^{(n)}) \rangle$, such that:

$$\exists x^{(0)}, \ldots, x^{(n)}. Init(x^{(0)}) \wedge Tr^n(x^{(0)}, \ldots, x^{(n)}) \wedge Bad(x^{(n)}).$$

b) Nontermination: A transition system is called **nonterminating** if there exists an initial state, from which it is possible to take any number of transitions:

$$\exists x. Init(x) \wedge \forall n \in \mathbb{N}. \exists x^{(1)}, \ldots, x^{(n)}. Tr^n(x, x^{(1)}, \ldots, x^{(n)}).$$

Definition 1 (Recurrent Set)**.** For a Transition System $\langle Init, Tr, x \rangle$, a formula $N(x)$ is a recurrent set if: (1) $\forall x. N(x) \rightarrow \exists x'. Tr(x, x') \wedge N(x')$, (2) $\exists x. N(x) \wedge Init(x)$.

Transition System is nonterminating if there exists a *recurrent set*. Intuitively, every state in the recurrent set has a successor that also lies in the recurrent set, and the recurrent set contains some of the initial states.

c) Termination: A TS is called **terminating** if for every initial state it is possible to take only limited number of transitions. For a Transition System $\langle Init, Tr, x \rangle$:

$$\forall x. Init(x) \rightarrow \exists n \in \mathbb{N}. \neg \exists x^{(1)}, \ldots, x^{(n)}. Tr^n(x, x^{(1)}, \ldots, x^{(n)}).$$

Definition 2 (Transition invariant)**.** For a Transition System $\langle Init, Tr, x \rangle$, $TrInv(x, x')$ is a transition invariant if: $\forall x, x'. R(x) \wedge Tr^*(x, x') \rightarrow TrInv(x, x')$, where $Tr^*(x, x')$ is the reflexive transitive closure of $Tr(x, x')$ and $R(x)$ is a formula encoding all the states, reachable within transition system.

Definition 3 (Well-founded Relation)**.** A binary relation $R \subseteq D \times D$ over a domain D is *well-founded* if there exists no infinite descending chain:

$$\nexists\, d_0, d_1, d_2, \cdots \in D.\ \forall i \in \mathbb{N}.\ (d_i, d_{i+1}) \in R$$

A relation $R(x, x')$ can be proven to be well-founded [3] by synthesizing a ranking function $f(x)$, such that $\forall x, x'. R(x, x') \rightarrow f(x) > f(x')$.

Transition invariant [4] can be used to prove termination. If transition invariant is a finite union of well-founded relations (for example if every separate relation has a ranking function), then the Transition System terminates.

Definition 4 (Disjunctively well-founded relation)**.** A relation $D(x, x') = \cup_{i=0}^{n} R_i(x, x')$ is called disjunctively well-founded if every relation $R_i(x, x')$ is well-founded.

https://doi.org/

This article is licensed under a Creative Commons Attribution 4.0 International License

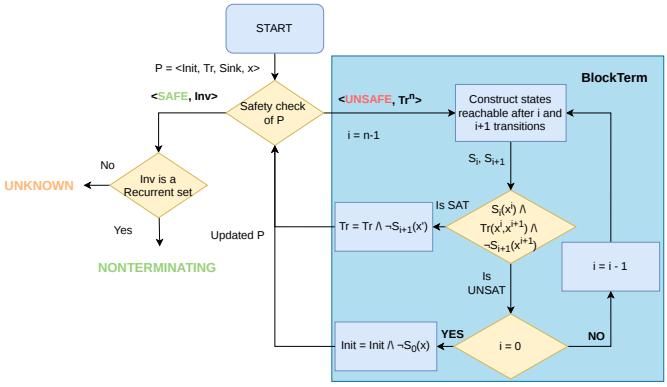


Figure 1. Execution flow of Safety-based Nontermination analysis, cf [1]

Algorithm 1: Nontermination analysis using Safety, cf [1]

Input : Safety problem $P = \langle Init, Tr, Sink, x\rangle$
Output: NONTERM | UNKNOWN

while $\top$ **do**
  $\langle res, \phi = Tr^n \text{ or } Inv\rangle \leftarrow safetyCheck(P)$
  if $res = UNSAFE$ **then**
    $P \leftarrow$ BLOCKTERM(P, ϕ)
  else
    if $\forall x.\phi(x) \rightarrow \exists x'.Tr(x, x') \wedge \phi(x')$ **then**
      return NONTERM
    return UNKNOWN

Explain what does unsat? means

d) Craig Interpolation: An abstraction technique [2], such that given two formulas A and B such that $A \wedge B$ is unsatisfiable, a Craig interpolant for A and B is a formula I such that $A \rightarrow I$, $I \wedge B$ is unsatisfiable, and I only contains the symbols, common to both A and B. We use $Itp(A, B)$ to denote the computation of the Craig interpolant for A and B.

A. Proving Nontermination via Safety

Many techniques reduce termination and nontermination analysis to safety verification. One notable approach is the reduction of nontermination to safety, which constructs strong recurrent sets via inductive invariants [1]. It was introduced in the context of proving nontermination of programs with loops, and can be extended for the Transition Systems. This approach is described below.

Definition 5 (Sink states)**.** For a Transition System $\langle Init, Tr, x\rangle$, a state s is a sink state, if $\neg\exists s'.Tr(s, s')$. We use $Sink(x)$ to denote the formula encoding all sink states in the transition system, such that $\forall x.Sink(x) \leftrightarrow \neg\exists x'.Tr(x, x')$

The Algorithm 1 for Safety-based Nontermination analysis takes a safety problem $P = \langle Init, Tr, Sink, x\rangle$, where $Sink(x) = QE(\exists x'.\neg Tr(x, x'))$. As an output, it returns NONTERM if the system is nonterminating, and UNKNOWN, if the algorithm cannot conclude the analysis. The Figure 1 contains the execution flow of Safety-based Nontermination analysis.

The approach executes a safety check of P in a loop. At line 2, the algorithm handles the case when P is unsafe (i.e., exists a trace reaching sink states). In this case, the algorithm calls the BLOCKTERM procedure (Algorithm 2), which refines the safety problem P by blocking the states that determenistically lead to the sink states, updating the Safety problem. BLOCKTERM is described in detail below. In the case when P is safe, algorithm checks if inductive invariant $Inv(x)$ is a recurrent set (line 5). If $Inv(x)$ is a recurrent set, then the transition system is nonterminating. Otherwise, algorithm returns UNKNOWN, as the analysis is inconclusive.

Add data field to the algorithm and algos below

Algorithm 2: BLOCKTERM (P, Tr^n), cf [1]

Input : Safety problem $P = \langle Init, Tr, Sink, x\rangle$,
terminating trace $Tr^n(x^{(0)}, \ldots, x^{(n)}) = Tr(x^{(0)}, x^{(1)}) \wedge \cdots \wedge Tr(x^{(n-1)}, x^{(n)})$
Output: Refined safety problem $P' = \langle Init', Tr', Sink', x\rangle$

1 $i \leftarrow n - 1$
2 **while** $i > 0$ **do**
3 $S_{i+1}(x^{(i+1)}) \leftarrow QE(\exists x^{(0)}, \ldots, x^{(i)}, x^{(i+2)}, \ldots, x^{(n)}.$
  $Init(x^{(0)}) \wedge Tr^n \wedge Sink(x^{(n)}))$
4 $S_i(x^{(i)}) \leftarrow QE(\exists x^{(0)}, \ldots, x^{(i-1)}, x^{(i+1)}, \ldots, x^{(n)}.$
  $Init(x^{(0)}) \wedge Tr^n \wedge Sink(x^{(n)}))$
5 **if** *SAT ?* $S_i(x) \wedge Tr(x, x') \wedge \neg S_{i+1}(x')$ **then**
6 $Tr'(x, x') \leftarrow Tr(x, x') \wedge \neg S_{i+1}(x')$
7 **return** $\langle Init, Tr', Sink, x\rangle$
8 $i \leftarrow i - 1$
9 $Init'(x^{(0)}) \leftarrow Init(x^{(0)}) \wedge$
  $\neg QE(\exists x^{(1)}, \ldots, x^{(n)}.Init(x^{(0)}) \wedge Tr^n \wedge Sink(x^{(n)}))$
10 **return** $\langle Init', Tr, Sink, x\rangle$

The BLOCKTERM procedure takes as input a safety problem P and a trace formula $Tr^n(x^{(0)}, \ldots, x^{(n)})$ such that $Init(x) \wedge Tr^n(x, \ldots, x') \wedge Sink(x')$ is satisfiable. The algorithm iteratively analyses the transition trace from the sink. It checks if it is possible to take a transition that does not lead to the sink states(at line 4), using a query to the SMT solver. If it is possible, then the algorithm blocks the states that deterministically lead to the sink states at line 6. Otherwise, if such states do not exist, the algorithm blocks the subset of initial states that deterministically lead to the sink states (line 9).

Lemma 1. *If Algorithm 1 returns NONTERM, then the transition system is nonterminating, cf [3].*

Example 1. Consider a transition system $TS = \langle Init, Tr, x\rangle$ with boolean variables $x = \{a, b, c\}$ and: $Init(x) \triangleq a$, $Tr(x, x') \triangleq (a \wedge b') \vee (a \wedge c') \vee (b \wedge c')$. The sink states are computed as $Sink(x) \triangleq c$. The initial safety problem is $P = \langle Init, Tr, Sink, x\rangle$.

Iteration 1. $safetyCheck(P)$ returns UNSAFE with the trace $Tr^1(x^{(0)}, x^{(1)})$ corresponding to the path $a \rightarrow c$, i.e., $Init(x^{(0)}) \wedge Tr(x^{(0)}, x^{(1)}) \wedge Sink(x^{(1)})$. BLOCKTERM analyses the trace and detects that from state a there exists an alternative successor, namely b, that does not lead directly to $Sink$. Formally, the check $S_0(x) \wedge Tr(x, x') \wedge \neg S_1(x')$ is

satisfiable (with $S_1 \equiv c$), so the transition relation is refined:

$$Tr'(x, x') \leftarrow Tr(x, x') \wedge \neg c'$$

Iteration 2. $safetyCheck(\langle Init, Tr', Sink, x\rangle)$ returns SAFE with an inductive invariant $Inv(x)$. The algorithm checks whether $Inv(x)$ is a recurrent set. Since Tr' blocks the transition to c, state b has no outgoing transition under Tr', so Inv is not a recurrent set. As no recurrent set was found, Algorithm 1 returns $UNKNOWN$.

II. Termination Analysis via Interpolation-based Transition Invariant Generation

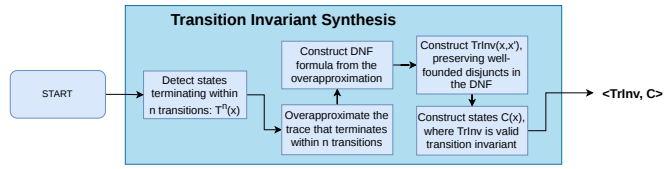
We propose a new approach for proving termination that leverages safety verification to generate a well-founded transition invariant candidate via interpolation. At each iteration, our procedure constructs a trace from the initial states to a sink state. A well-founded transition invariant is then constructed based on the overapproximation of the terminating trace. If the invariant covers all reachable states, termination is concluded. Additionally, the algorithm uses the candidate transition invariant to refine the state space, focusing on states where transition invariant is not valid. The interpolation-based approach enables efficient property-directed transition invariant generation, as it is constructed with respect to the termination condition.

A. Interpolation-based Generation of Well-Founded Transition Invariants

The core of our approach is the procedure for automated, interpolation-based generation of well-founded transition invariants: TrInvSynthesis. Given a transition system and a terminating trace, the procedure constructs a disjunctively well-founded approximation of the transition relation. If the resulting formula constitutes a transition invariant over all reachable states, it witnesses termination.

The Procedure consists of five major steps: (1) computing T^n, the formula encoding states that terminate within n transitions. T^n is used to construct the interpolant. It guarantees that no states other then $Sink$ are reachable in n transitions; (2) constructing an over-approximation of the terminating trace via interpolation. The usage of sink states allows the interpolant to capture the reason for termination; (3) converting the interpolant into an underapproximating DNF formula. This allows to simplify synthesis of the ranking function for the disjuncts, as every disjunct is a conjunction; (4) filtering disjuncts to retain only those encoding well-founded relations. Transition invariant needs to be disjunctively well-founded to witness termination; and (5) computing the formula encoding states for which the produced formula is a valid transition invariant. Transition invariant needs to be valid only over the reachable states to conclude termination, which weakens the properties that need to hold for transition invariant. The overall flow of the procedure is illustrated in the Figure 2.

Algorithm 3 takes as an input a safety problem $P = \langle Init, Tr, Sink, x\rangle$, a transition trace $Tr^n(x^{(0)}, x^{(1)}, \ldots, x^{(n)})$, and a transition invariant formula



Too much text in the diagram

Figure 2. Execution Flow of Interpolation-based Transition Invariant Generation.

Algorithm 3: TrInvSynthesis$(P, Tr^n, TInv)$.

Input : Safety problem $P = \langle Init, Tr, Sink, x\rangle$, transition trace $Tr^n(x^{(0)}, x^{(1)}, \ldots, x^{(n)})$, candidate transition invariant $TInv(x, x')$

Output: Updated transition invariant $TInv(x, x')$ and formula $C(x)$ encoding states covered by transition invariant

1 $T^n(x^{(0)}) \leftarrow \neg QE(\exists x^{(1)}, \ldots, x^{(n)}.$
$\quad Tr^n(x^{(0)}, x^{(1)}, \ldots, x^{(n)}) \wedge \neg Sink(x^{(n)}))$

2 $itp(x, x') \leftarrow Itp(Tr^n(x, x^{(1)}, \ldots, x'), T^n(x) \wedge \neg Sink(x'))$

3 $Cs(x, x') \leftarrow toDNF(itp)$

4 $TInv \leftarrow TInv \vee \bigvee_i \{d_i \in Cs | \exists$ ranking function $f(x)\}$

5 $C(x) \leftarrow \neg QE(\exists x', x''.$
$\quad Tr(x, x') \wedge \big(Id(x', x'') \vee TInv(x', x'')\big) \wedge \neg TInv(x, x''))$

6 **return** $\langle TInv, C\rangle$

$TInv(x, x')$. It returns a tuple of formulas $\langle TInv, C\rangle$, where $TInv(x, x')$ is a disjunctively well-founded transition invariant candidate, and $C(x)$ is a formula encoding the states for which $TInv$ is a valid transition invariant. It proceeds in the following steps:

a) Step 1: Computing T^n (line 1). $T^n(x)$ encodes all states guaranteed to reach a sink state within n transitions, is defined by the following property:

$$\forall x^{(0)}, \ldots, x^{(n)}. T^n(x^{(0)}) \wedge Tr^n(x^{(0)}, \ldots, x^{(n)}) \rightarrow Sink(x^{(n)})$$

T^n is constructed with quantifier elimination, eliminating all variables from the formula except for $x^{(0)}$. Construction $T^{(n)}$ enables the overapproximation of the trace.

Example 2. Consider $TS = \langle Init, Tr, x\rangle$ with $Init \triangleq v > 0$, $Tr(x, x') \triangleq ((v' = v - 1 \vee v' = v - 2) \wedge v > 0) \vee (v < -10 \wedge v' = v - 1)$, $x \triangleq \{v\}$. For $n = 2$ quantifier elimination would yield $T^2(x) = v \leq 2$. States with $v \in \{3, 4\}$ can reach $Sink$ in two transitions but are not guaranteed to do so, so they are not included in T^2. States with $v \in \{1, 2\}$ terminate in 1 or 2 transitions.

b) Step 2: Interpolation (line 2). The following formula is unsatisfiable by construction, since any state in T^n must reach $Sink$ within n steps:

$$T^n(x^{(0)}) \wedge Tr^n(x^{(0)}, \ldots, x^{(n)}) \wedge \neg Sink(x^{(n)})$$

This allows to compute the **Craig interpolant**, which over-approximates the trace Tr^n. Crucially, the usage of the sink states for interpolant construction allows the interpolant to

capture the reason *why* the particular trace is leading to the termination.

c) Step 3: DNF underapproximation (line 3). For transition invariant to be a proof of termination, it must be disjunctively well-founded, meaning every disjunct must encode a well-founded relation. Full conversion to a DNF format is expensive and can cause an exponential blowup in formula size. Additionally, even full DNF conversion does not guarantee that the resulting formula would witness termination, or even be a transition invariant. Our algorithm follows a lazy approach to DNF conversion, constructing an underapproximating DNF formula on demand, limiting the number of disjuncts in the resulting formula by a tunable constant. The disjuncts are constructed via syntactic processing of the formula, unwrapping the innermost disjunctions and conjunctions. [**Konstantin**: *To be slightly rewritten.*] It allows to significantly reduce the size of the resulting formula and speed up the DNF conversion, while preserving correctness: in the worst case, the resulting formula fails to be a transition invariant. Empirically, we observed that limiting the number of disjuncts results in more size-efficient and robust termination proofs.

Rwrite and fix the writing.
Say why we are still better compared to
mulas!!! - at least what is expected and how it is produced

d) Step 4: Filtering Well-Founded Disjuncts (line 4). For each disjunct $d(x, x') \in Cands(x, x')$ algorithm checks if $d(x, x')$ encodes a well-founded relation. Our technique uses automated linear ranking function synthesis [3] for checking the existence of a ranking function. The disjuncts for which a ranking function exists are preserved, and the rest are discarded. The resulting formula is disjoined with the input candidate transition invariant $TInv$.

e) Step 5: Synthesis of the states for which transition invariant is valid (line 5). If $TrInv$ is a transition invariant over all reachable states, then TS terminates, since every disjunct corresponds to well-founded relation. To verify this, procedure computes the formula $C(x)$ encoding the states over which $TrInv$ is a valid transition invariant.

Definition 6 (Coverage formula)**.** Consider a transition system $TS = \langle Init, Tr, x\rangle$ and a formula $TrInv(x, x')$. We say that $C(x)$ is a coverage formula, if the following property holds:

$$\forall x, x', x''.C(x) \wedge (TrInv(x, x') \vee Id(x, x')) \wedge Tr(x', x'') \rightarrow TrInv(x, x'')$$

Meaning, that $TrInv$ is a transition invariant over states satisfying $C(x)$.

Similarly to T^n, $C(x)$ is constructed using the quantifier elimination, negating the states for which $TrInv$ is not a transition invariant.

Example 3. Continuing Example 2, suppose the procedure constructs $TrInv \triangleq v' < v \wedge v' \geq 0$. Quantifier elimination then yields $C(x) \equiv v \geq 0$.

Note. If $\neg C(x)$ is not reachable, then $TrInv(x, x')$ is a disjunctively well-founded transition invariant over all reachable states, and witnesses the termination of transition system. For illustration, transition invariant in the Example 3 is sufficient to prove termination, since $\neg C(x) \triangleq v < 0$ is not reachable in the transition system.

Lemma 2. *Let $TS = \langle Init, Tr, x\rangle$ be a transition system and let $C(x)$ be a formula such that $\neg C(x)$ is unreachable in TS. If $TrInv(x, x')$ is a finite disjunction of formulas encoding well-founded relations and satisfies:*

$$\forall x, x', x''.C(x) \wedge (TrInv(x, x') \vee Id(x, x')) \wedge Tr(x', x'') \rightarrow TrInv(x, x'')$$

then TS is terminating.

Proof. We show that $TrInv$ is a disjunctively well-founded transition invariant over the reachable states of T, from which termination follows by [4].

Well-foundedness. By construction, for each disjunct d_i of $TrInv = \bigvee_i d_i$ there exists a ranking function. Therefore, every disjunct is a logical encoding of well-founded relation [3]. Hence $TrInv$ is disjunctively well-founded.

Transition invariant. We show that $\forall x, x'.\ R(x) \wedge Tr^*(x, x') \rightarrow TrInv(x, x')$, where $R(x)$ denotes the reachable states of T, by induction on the number of steps n in $Tr^*(x, x')$.

Base case ($n = 1$): It is needed to prove that:

$$R(x) \wedge Tr(x, x') \rightarrow TrInv(x, x')$$

Since $\neg C(x)$ is unreachable, $R(x) \rightarrow C(x)$.

The following assumption holds originally by the lemma statement:

$$C(x) \wedge Id(x, x') \wedge Tr(x', x'') \rightarrow TrInv(x, x'')$$

Since $Id(x, x')$ is an identity relation, it can be simplified to:

$$C(x) \wedge Tr(x, x') \rightarrow TrInv(x, x').$$

Since $R(x) \rightarrow C(x)$, therefore $R(x) \wedge Tr(x, x') \rightarrow C(x) \wedge Tr(x, x')$, the base case holds.

Inductive step: Assume $R(x) \wedge Tr^n(x, \ldots, x') \rightarrow TrInv(x, x')$ for some $n \geq 1$. Let $Tr^{n+1}(x, x'')$ hold, so there $\exists x, \ldots, x', x''.R(x) \wedge Tr^n(x, \ldots, x') \wedge Tr(x', x'')$. Since $R(x) \wedge Tr^n(x, \ldots, x') \rightarrow TrInv(x, x')$, then trivially $R(x) \wedge Tr^n(x, x') \wedge Tr(x', x'') \rightarrow R(x) \wedge TrInv(x, x') \wedge Tr(x', x'')$. Now, due to the fact that $R(x) \rightarrow C(x)$, the following holds:

$$R(x) \wedge TrInv(x, x') \wedge Tr(x', x'') \rightarrow C(x) \wedge TrInv(x, x') \wedge Tr(x', x'').$$

And by claim in the lemma:

$$C(x) \wedge TrInv(x, x') \wedge Tr(x', x'') \rightarrow TrInv(x, x'').$$

Therefore, through the chain of implications, we have:

$$R(x) \wedge Tr^{n+1}(x, x') \rightarrow TrInv(x, x').$$

Thus $TrInv$ is a disjunctively well-founded transition invariant over all reachable states, and TS is terminating. $\square$

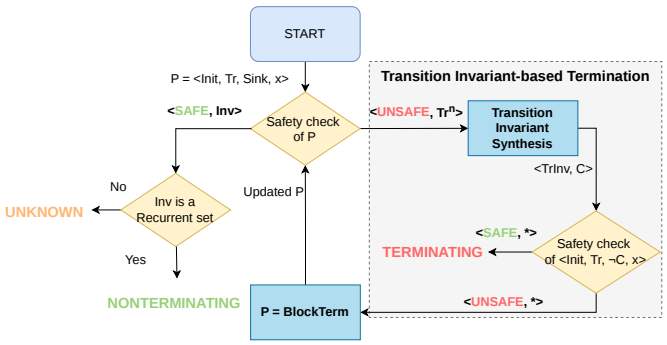


Figure 3. Execution Flow of Interpolation-based Transition Invariant Generation (New components are placed in a grey block).

State what lines were changed

Algorithm 4: Interpolation-based Termination Analysis.

Input : Safety problem $P = \langle Init, Tr, Sink, x\rangle$
Output: TERM | NONTERM | UNKNOWN

1 $TInv \leftarrow \bot$
2 **while** $\top$ **do**
3 $\quad \langle res, \phi = Tr^n \text{ or } Inv\rangle \leftarrow safetyCheck(P)$
4 $\quad$ **if** $\langle res, \phi\rangle = \langle UNSAFE, Tr^n(x^{(0)}, \ldots, x^{(n)})\rangle$ **then**
5 $\quad\quad \langle TInv, C\rangle \leftarrow$ TRINVSYNTHESIS$(P, Tr^n, TInv)$
6 $\quad\quad \langle res', \phi'\rangle \leftarrow safetyCheck(\langle Init, Tr, \neg C, x\rangle)$
7 $\quad\quad$ **if** $res' = UNSAFE$ **then**
8 $\quad\quad\quad P \leftarrow$ BLOCKTERM(P, Tr^n)
9 $\quad\quad$ **else**
10 $\quad\quad\quad$ **return** $TERM$
11 $\quad$ **else**
12 $\quad\quad$ **if** $\forall x. Inv(x) \rightarrow \exists x'. Tr(x, x') \wedge Inv(x')$ **then** **return** $NONTERM$
13 $\quad\quad$ **return** UNKNOWN

B. Interpolation-based Termination Analysis

This subsection presents the full termination analysis procedure, using the interpolation-based transition invariant generation. It integrates TRINVSYNTHESIS into the safety-based analysis loop of Algorithm 1. At each iteration, instead of only blocking terminating states, the algorithm also constructs a transition invariant candidate. The overall flow is shown in Figure 3.

The Algorithm 4 takes as input a Safety Problem $P = \langle Init, Tr, Sink, x\rangle$, where $Init$ and Tr encode the initial states and transition relation, $Sink$ is the formula encoding sink states, and x is a set of variables over which the Transition System is defined. As an output, the algorithm returns NONTERM if TS is nonterminating, TERM if it is terminating, and UNKNOWN if the procedure cannot determin the final result. The procedure maintains a formula encoding a candidate transition invariant $TInv(x, x')$. During the execution, $TInv$ is extended with new disjuncts, which are generated by the TRINVSYNTHESIS procedure, and encode well-founded relations.

Algorithm commences by analyzing the original safety problem. Every iteration, similarly to the Algorithm 1, begins with the safety check at line 2. The algorithm handles two possible cases:

a) UNSAFE.: A terminating trace Tr^n exists. The algorithm calls TRINVSYNTHESIS to extend $TInv$ and compute the coverage formula $C(x)$ (line 5). Then, it checks if $\neg C(x)$ is reachable from the initial states (line 5). If $\neg C(x)$ is reachable, algorithm refines the safety problem by blocking the states deterministically leading to termination via BLOCKTERM, and the loop continues. If it is not reachable, then $TInv$ is a transition invariant over all reachable states, and algorithm concludes termination.

b) SAFE.: No sink state is reachable. If $Inv(x)$ is a recurrent set, the system is nonterminating. Otherwise, the algorithm returns UNKNOWN as the analysis is inconclusive.

Example 4. Consider TS from Example 2 with $Init \triangleq v > 0$, $Tr \triangleq \big((v' = v - 1 \vee v' = v - 2) \wedge v > 0\big) \vee \big(v < -10 \wedge v' = v - 1\big)$, $x \triangleq \{v\}$.

Preprocessing computes $Sink \equiv v \leq 0 \wedge v \geq -10$. Then, the algorithm is called with $P = \langle Init, Tr, Sink, x\rangle$.

The first safety check detects that state satisfying $Sink$ exist within $Init(x)$. Constructed trace is empty, therefore constructed transition invariant and covered states are $TInv \triangleq \bot$ and $C(x) \triangleq \bot$. Algorithm proceeds to BLOCKTERM, updating init states:

$$Init(x) \leftarrow v > 0 \vee v < -10$$

On the next iteration, a trace Tr^1 reaching $Sink$ is found. TRINVSYNTHESIS produces a transition invariant candidate $TInv(x, x') \triangleq v' \leq v - 1 \wedge v' \geq 0$ and the formula $C(x) = v \geq 0$. The safety check for $\langle Init, Tr, \neg C(x), v\rangle$ returns SAFE, and algorithm concludes that the transition system is terminating.

C. Correctness

This subsection proves the soundness of the introduced approach. We first prove that when algorithm returns TERM, the transition system is nonterminating, and then we prove the correctness of the nontermination.

Lemma 3. *If Algorithm 4 returns $TERM$, then the transition system $\langle Init, Tr, x\rangle$ is terminating.*

Proof. $TERM$ is returned only in the line 10, of the algorithm. It means that there exists a transition invariant $TInv$ which is valid over states $C(x)$, and $\neg C(x)$ is unreachable from the initial states. The safety problem is updated iteratively, blocking the states that deterministically lead to the termination, so in the end the $P = \langle Init', Tr', Sink, x\rangle$, where $Init'(x) = Init(x) \wedge \bigwedge_{i=0}^{m} \neg S_i(x)$ and $Tr' = Tr(x) \wedge \bigwedge_{i=0}^{l} \neg S_j(x')$. According to the Lemma 2, Transition System $\langle Init', Tr', x\rangle$ is terminating, since $TInv$ is a disjunctively well-founded transition invariant over all reachable states.

Since the states satisfying $\bigvee_{i=0}^{m} S_i(x)$ and $\bigvee_{i=0}^{l} S_j(x')$ deterministically lead to the termination, every trace of $\langle Init, Tr, x\rangle$ either follows Tr' or passes through a blocked state which leads to termination. Hence $\langle Init, Tr, x\rangle$ is terminating. $\square$

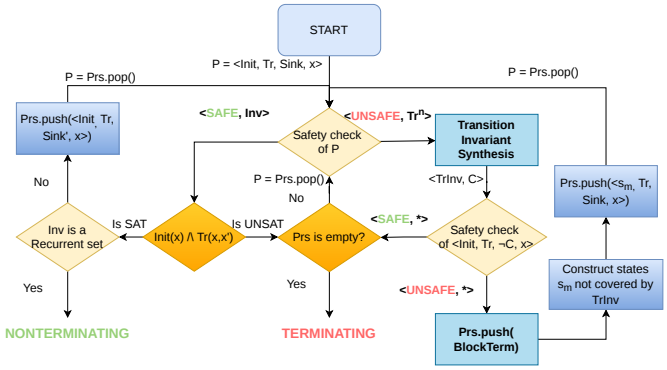


Figure 4. Optimized Interpolation-based Termination Analysis (New components are highlighted in dark orange and dark blue colours).

Lemma 4. *If Algorithm 4 returns $NONTERM$, then the transition system $\langle Init, Tr, x\rangle$ is nonterminating.*

Proof. The proof of this lemma follows directly from the lemma 1, as the nontermination proof procedure directly corresponds to one described in the Algorithm 1. □

III. Interpolation-based Termination Analysis: Optimizations

Algorithm 4 is already capable to prove termination efficiently, via the construction of transition invariants. This section describes three optimizations that extend its power: a termination check, safety problem refinement, and targeted analysis of states not covered by the transition invariant. The overall flow of the optimized algorithm is shown in Figure 4.

Reference the actual lines in the algorithm

A. Termination Check

When the safety check returns SAFE and $Inv(x)$ is not a recurrent set, the algorithm can additionally check whether it is possible to take transition from the initial states by querying:

$$Init(x) \wedge Tr(x, x')$$

If this formula is unsatisfiable, no transition can be taken from any initial state (or all transitions are blocked, meaning all reachable states deterministically lead to termination) and the system is trivially terminating.

B. Safety Problem Refinement

When $Inv(x)$ is not a recurrent set, this is often because BlockTerm has eliminated all transitions leading to $Sink$, leaving states in Inv with no outgoing transition. These states are themselves sink states under the refined Tr' and can be computed as:

$$Sink'(x) = \neg QE(\exists x'.Tr'(x, x'))$$

The formula $Sink'$ replaces $Sink$ in the safety problem, enabling the analysis to continue rather than returning UNKNOWN.

Algorithm 5: Optimized Interpolation-based Termination Analysis.

Input : Safety problem $P = \langle Init, Tr, Sink, x\rangle$
Output: TERM | NONTERM

1 $TInv \leftarrow \bot$
2 $Prs \leftarrow [P]$
3 **while** $\top$ **do**
4 $\quad \langle Init', Tr', Sink', x\rangle \leftarrow Prs.pop()$
5 $\quad \langle res, \phi\rangle \leftarrow safetyCheck(\langle Init', Tr', Sink', x\rangle)$
6 $\quad$ **if** $\langle res, \phi\rangle = \langle UNSAFE, Tr^n(x^{(0)}, \ldots, x^{(n)})\rangle$ **then**
7 $\quad\quad \langle TInv, C\rangle \leftarrow$ TrInvSynthesis$(P, Tr^n, TInv)$
8 $\quad\quad \langle res', \phi'\rangle \leftarrow safetyCheck(\langle Init', Tr', \neg C, x\rangle)$
9 $\quad\quad$ **if** $\langle SAFE, Inv(x)\rangle$ **then**
10 $\quad\quad\quad$ **if** $Prs = \emptyset$ **then return** $TERM$
11 $\quad\quad$ **else**
12 $\quad\quad\quad Prs.push($BlockTerm$(P', Tr^n))$
13 $\quad\quad\quad S_m \leftarrow QE(\exists x^{(0)}, \ldots, x^{(m-1)}, Init(x^{(0)}) \wedge$
$\quad\quad\quad\quad Tr^n(x^{(0)}, \ldots, x^{(m)}) \wedge \neg C(x^{(m)}))$
14 $\quad\quad\quad Prs.push(\langle S_m, Tr', Sink', x\rangle)$
15 $\quad$ **else**
16 $\quad\quad$ **if** $UNSAT\ ?\ Init(x) \wedge Tr(x, x')$ **then return** $TERM$
17 $\quad\quad$ **if** $\forall x.Inv(x) \rightarrow \exists x'.Tr(x, x') \wedge Inv(x')$ **then return** $NONTERM$
18 $\quad\quad Sink'(x) \leftarrow \neg QE(\exists x'.Tr(x, x'))$
19 $\quad\quad Prs.push(\langle Init', Tr', Sink', x\rangle)$

C. Refinement of transition invariant

Constructed transition invariant can be used to guide the termination analysis. Particularly, the algorithm constructs the formula $C(x)$ encoding states for which the transition invariant is valid. If $\neg C(x)$ is reachable, it means that there exist states for which the transition invariant is not valid, and which potentially can lead to non-termination. The intuition is to compute the reachable states S_m within $\neg C(x)$ to construct transition invariant covering those states in particular, or to detect a recurrent set otherwise.

D. Full Algorithm

Algorithm 5 extends Algorithm 4 with three changes corresponding to the optimizations above. First, in addition to the transition invariant, the algorithm maintains a list of safety problems which need to be analyzed: Prs (line 2). Each iteration, the algorithm pops a safety problem from the list and analyzes it. This allows the algorithm to concentrate on the termination analysis of particular reachable states within TS, to efficiently expand the transition invariant. The original safety problem is always present in the Prs (with blocked states that deterministically lead to termination).

Second, when $\neg C$ is reachable, the algorithm pushes both the BlockTerm-refined problem and a new problem focused on the states, not covered by transition invariant S_m (line 14). Thus allowing to concentrate on the termination analysis of particular reachable states, instead of necessarily proving termination for the whole system.

Third, additional changes were made in the SAFE case, where algorithm checks for the termination at line 16, and

refines the safety problem with the new sink states instead of returning UNKNOWN (line 18).

Lemma 5. *If Algorithm 5 returns $TERM$, then the original transition system $\langle Init, Tr, x\rangle$ is also terminating.*

Proof. TERM is returned at line 10 or line 16. By construction, to return $TERM$, the termination conditions need to be satisfied for the transition system $\langle Init', Tr', x\rangle$, where $Init'(x) = Init(x) \wedge \bigwedge_i \neg S_i(x)$, and $Tr'(x, x') = Tr(x, x') \wedge \bigwedge_j \neg S_j(x')$. $\bigvee_{i=0}^{l} S_i(x)$ and $\bigvee_{j=0}^{k} S_j(x')$ are the states that deterministically lead to termination.

Case 1. If $TERM$ is returned at line 10, then the termination proof follows directly from the Lemma 2.

Case 2. If $TERM$ is returned at line 16, then the formula $Init'(x) \wedge Tr'(x, x')$ is unsatisfiable, so no transition is reachable from $Init'$. It means that the transition system is terminating, since all feasible transitions in the original TS lead to deterministically terminating states. □

Lemma 6. *If Algorithm 5 returns $NONTERM$, then the original transition system $\langle Init, Tr, x\rangle$ is nonterminating.*

Proof. NONTERM is returned at line 17, where $Inv(x)$ is a recurrent set for the current refined system $\langle Init', Tr', x\rangle$. By construction, $Init'$ is reachable from $Init$ (it is either $Init$ itself or derived from a trace starting in $Init$), and $Tr'(x, x') \implies Tr(x, x')$. Based on the Lemma 1, if $NONTERM$ is returned, then there exists a recurrent set in the transition system $\langle Init', Tr', x\rangle$, which means that there also exists a recurrent set in the original transition system $\langle Init, Tr, x\rangle$. It proves nontermination. □

References

[1] Chen, H.Y., Cook, B., Fuhs, C., Nimkar, K., O'Hearn, P.W.: Proving nontermination via safety. In: Ábrahám, E., Havelund, K. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings. pp. 156–171. Lecture Notes in Computer Science, Springer (2014). https://doi.org/10.1007/978-3-642-54862-8_11, https://doi.org/10.1007/978-3-642-54862-8_11

[2] Craig, W.: Three uses of the Herbrand-Gentzen theorem in relating model theory and proof theory. The Journal of Symbolic Logic **22**(3), 269–285 (1957)

[3] Podelski, A., Rybalchenko, A.: A complete method for the synthesis of linear ranking functions. In: Steffen, B., Levi, G. (eds.) Verification, Model Checking, and Abstract Interpretation, 5th International Conference, VMCAI 2004, Venice, Italy, January 11-13, 2004, Proceedings. pp. 239–251. Lecture Notes in Computer Science, Springer (2004). https://doi.org/10.1007/978-3-540-24622-0_20, https://doi.org/10.1007/978-3-540-24622-0_20

[4] Podelski, A., Rybalchenko, A.: Transition invariants. In: 19th IEEE Symposium on Logic in Computer Science (LICS 2004), 14-17 July 2004, Turku, Finland, Proceedings. pp. 32–41. IEEE Computer Society (2004). https://doi.org/10.1109/LICS.2004.1319598, https://doi.org/10.1109/LICS.2004.1319598